翁恺课 · 第 8 主题

W8 指针与字符串

对齐翁恺《程序设计入门——C语言》第 8 讲 · 目标：明白指针是"变量的门牌号"（& 取地址 / * 解引用）、会用指针作参数真正改外部变量（弥补 W6 的 swap 遗憾）、认识字符串是"以 '\0' 结尾的字符数组" · 不覆盖：动态内存、函数指针、结构体（后续进阶）

8.1 为什么需要指针：变量的"门牌号"

回忆 W6 那个遗憾：`swap(x, y)` 传值换不动——因为函数拿到的是**值的复制**。想真正改外面的变量，就得告诉函数"这个变量**住在哪个地址**"。翁恺的说法：**指针就是变量的"门牌号"**。变量在内存里有个地址（就像门牌号），指针变量存的正是这个门牌号；顺着门牌号（解引用）就能找到并改那间"房间"里的东西。

> **一句话：**普通变量是"值"，指针是"地址"。& 取地址（拿门牌号），* 解引用（进房间）。这是 C 最难也最强大的一关，学完它 W6 的 swap 就解了。

8.2 指针初识：& 取地址、* 解引用

声明指针用 `类型 *名字`（如 `int *p` 表示"p 指向一个 int"）。`&x` 拿到 x 的地址，`*p` 顺着 p 找到它指向的变量：

```
/* wk8_addr.c — 指针初识：& 取地址，* 解引用（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int x = 5;
    int *p = &x;          /* p 存 x 的地址 */
    printf("x = %d, *p = %d\n", x, *p);
    *p = 10;              /* 通过指针改 x */
    printf("改后 x = %d\n", x);
    return 0;
}
```

实跑输出：

```
x = 5, *p = 5
改后 x = 10
```



图 W8-1　指针 = 门牌号：& 取到 x 的地址存进 p，*p 顺着门牌号进房间拿/改值

8.3 指针作参数：swap 这次换得动

把地址传给函数，函数里用 *a、*b 改的就是**外面的原变量**——W6 那个换不动的 swap，现在用指针解了：

```
/* wk8_swap.c — 指针作参数：这次 swap 真正换得动（gcc 实跑）*/
#include <stdio.h>

void swap(int *a, int *b);

int main(void)
{
    int x = 3, y = 5;
    printf("调用前：x=%d y=%d\n", x, y);
    swap(&x, &y);          /* 传地址 */
    printf("调用后：x=%d y=%d\n", x, y);
    return 0;
}

void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

实跑输出：

```
调用前：x=3 y=5
调用后：x=5 y=3
```



图 W8-2　传值 vs 传址：W6 传值换的是副本、外面不变；W8 传地址（&x）通过指针 *a 改的就是原变量、swap 真正换得动

8.4 指针与数组：数组名就是首地址

数组名 `a` 就是数组**第一个元素的地址**（首地址）。所以 `int *p = a` 让 p 指向 a[0]，之后 `p++` 自动往后挪一个 int，`*p` 依次是 a[0]、a[1]……：

```c
/* wk8_ptr_array.c — 指针遍历数组：p 从首地址走到尾（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p;
    for (p = a; p < a + 5; p++) {
        printf("%d ", *p);
    }
    printf("\n");
    return 0;
}
```

实跑输出：

```
10 20 30 40 50
```

> **翁恺提醒：**之前学"数组作参数其实传的是首地址"（W6），现在知道为什么了——数组名本身就是地址。这也是为什么函数里改数组外面会变：函数拿到的就是那批数据**真实的门牌号**。

8.5 字符串初识：以 '\0' 结尾的字符数组

C 没有专门的"字符串"类型，字符串就是**字符数组**，且必须以 '\0'（**字符串结束符**）结尾。"Hello" 实际占 6 个格子（5 个字符 + 1 个 '\0'）。%s 从开头一直读到 '\0' 停：

```
/* wk8_str.c — 字符串：字符数组以 '\0' 结尾，%s 输出（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    char s[] = "Hello";          /* 实际存 6 个字符（含 '\0'）*/
    printf("%s\n", s);
    printf("第一个字符：%c\n", s[0]);
    return 0;
}
```

实跑输出：

```
Hello
第一个字符：H
```



图 W8-3　字符串内存布局："Hello" 是 5 个字符 + 1 个 '\0' 结束符；%s 靠 '\0' 知道在哪停

8.6 字符串操作：手写 strlen（数到 '\0' 停）

字符串处理最基础的就是**数长度**——标准库有 strlen，这里先手写一遍（理解"数到 '\0' 停"的本质）：

```
/* wk8_strlen.c — 手写 strlen：数到 '\0' 为止（gcc 实跑）*/
#include <stdio.h>

int my_strlen(char s[]);

int main(void)
{
    char s[] = "hello";
    printf("长度 = %d\n", my_strlen(s));
    return 0;
}

int my_strlen(char s[])
{
    int n = 0;
    while (s[n] != '\0') n++;   /* 数到结束符 */
    return n;
}
```

实跑输出：

```
长度 = 5
```

> **拓展：**标准库还常用 strcpy（复制）、strcmp（比较，按字典序，返回 0 表示相等）。它们的实现都依赖"以 '\0' 结尾"这个约定——所以忘写 '\0' 是最致命的错。

8.7 新手坑集中营（本讲最容易错的 3 件事）

1. **指针没指向就解引用（野指针）** → int *p; *p = 10; 写到一个**野地址**，可能崩溃或踩坏别的数据。gcc 实测给警告：warning: 'p' is used uninitialized [-Wuninitialized]。指针用前先让它**指向某处**（p = &x 或 p = a）。
2. **字符串忘留 '\0' 的空间** → char s[5] = "hello"; 只给 5 格装 6 个字符（含 '\0'），'\0' 无处安放会越界。gcc 实测给警告：warning: ... truncates NUL terminator ... [-Wunterminated-string-initialization]。装字符串要**多留一格**给 '\0'。
3. **数组名 ≠ 数组第一个元素的值** → 数组名是**首地址**，不是第一个元素本身。*a 才是 a[0]；a 直接当值用（如 a++）是错的（数组名不可改，它是常量地址）。

8.8 一分钟自测（答案见练习册）

判断/填空：

1. `int *p = &x;` 中 `&x` 表示____，`*p` 表示____；`*p = 10` 等价于____。
2. 数组名 `a` 是____（首地址/第一个元素）；`int *p = a` 后 `p++` 让 p 指向____。
3. 字符串 `"abc"` 实际占____个字符；靠____知道在哪结束；`char s[3] = "abc"` 会____（能/不能）正常用。

8.9 练习指引

去配套练习册做 W8 练习。翁恺建议本讲里程碑：**用指针把 swap、数组求和写一遍，再手写一个 strlen**——把"地址 + 解引用 + '\0' 约定"这三件事真正焊进脑子。做完这 8 个主题，你就握住了 C 语言最核心的变量、数组、函数、指针四块基石，可以开始刷题和上真机了。

本讲义为自写原创讲解（对齐翁恺《程序设计入门——C语言》第 8 讲），仅作课堂伴读；示例代码均用 gcc -Wall -Wextra 实测通过（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。视频与图片版权归原作者所有，本讲义不含课程原素材。